Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №4
з дисципліни
«Об’єктно-орієнтоване програмування»
на тему
«Вдосконалення структури коду графічного редактора об’єктів»

Виконав:
студент групи ІМ-53
Брагар Артем Сергійович
Номер в списку - 2

Перевірив:
Рекечинський Дмитро Олександрович

Київ 2026

Завдання

Студенти з непарним номером (1, 3, 5, . . .) програмують глобальний статичний об'єкт класу MyEditor.

3. Усі кольори та стилі (за винятком "гумового" сліду) геометричних форм – як у попередній лабораторній роботі №3. "Гумовий" слід при вводі усіх фігур малювати пунктирною лінією.

4. Окрім чотирьох типів фігур, які були у попередніх лаб. №2 та 3, запрограмувати ще введення та відображення двох нових фігур – лінія з кружечками та каркас куба.

Кольори ліній та заповнення цих нових фігур студент визначає на свій розсуд.

Для об’єктів типів лінії з кружечками та каркасу кубу відповідні класи запрограмувати саме множинним успадкуванням. Втім, якщо мова програмування не дозволяє множинне успадкування, то його можна замінити іншою конструкцією (наприклад, композицією).

У першу чергу це стосується метода Show для нових фігур – для відображення ліній треба використовувати виклики метода Show з класу LineShape, для відображення кружечків – виклики метода Show з класу EllipseShape, а для відображення прямокутників – виклики метода Show з класу RectShape.

5. Для усіх шести типів форм зробити кнопки Toolbar з підказками (tooltips)

6. У звіті повинна бути схема успадкування класів – **діаграма класів**. Для побудови діаграми класів дозволяється обрати інструменти на власний розсуд.

Лістинг коду
(Розширення файлів: .cs)
EllipseShape

```
namespace Lab4;
public class EllipseShape : Shape
{
public Brush FillBrush = new SolidBrush(Color.Cyan);
```

```
        public override void Show(Graphics g)
                                {
            int radX = Math.Abs(EndPoint.X - StartPoint.X);
            int radY = Math.Abs(EndPoint.Y - StartPoint.Y);
              int x = Math.Min(StartPoint.X, EndPoint.X);
              int y = Math.Min(StartPoint.Y, EndPoint.Y);
        if(!IsPreview) {g.FillEllipse(FillBrush, x, y, radX, radY);}
              g.DrawEllipse(OutlinePen, x, y, radX, radY);
                                }
                public override void Dispose()
                                {
                          FillBrush.Dispose();
                             base.Dispose();
                                }
                             }
```

LineShape

```
                        namespace Lab4;
                   public class LineShape : Shape
                                {
                     public override void Show(Graphics g)
                                {
              g.DrawLine(OutlinePen, StartPoint, EndPoint);
                                }
                                }
```

PointShape

```
                        namespace Lab4;
                    public class PointShape : Shape
                                {
                           public Brush FillBrush;
                             public PointShape()
                                {
             FillBrush = new SolidBrush(OutlinePen.Color);
                                }
                     public override void Show(Graphics g)
                                {
                               int width = 4;
                              int height = 4;
                  int x = StartPoint.X - (width / 2);
                 int y = StartPoint.Y - (height / 2);
            g.FillEllipse(FillBrush, x, y, width, height);
                                }
```

```
        public override void Dispose()
        {
            FillBrush.Dispose();
            base.Dispose();
        }
    }
```

RectShape

```
namespace Lab4;
public class RectShape : Shape
{
    public override void Show(Graphics g)
    {
        int dx = Math.Abs(EndPoint.X - StartPoint.X);
        int dy = Math.Abs(EndPoint.Y - StartPoint.Y);
        int width = dx * 2;
        int height = dy * 2;
        int x = StartPoint.X - dx;
        int y = StartPoint.Y - dy;
        g.DrawRectangle(OutlinePen, x, y, width, height);
    }
}
```

CubeShape

```
namespace Lab4;
public class CubeShape : Shape
{
    private RectShape frontRect = new RectShape();
    private RectShape backRect = new RectShape();
    private LineShape[] lineShapes = new LineShape[4];
    public CubeShape()
    {
        for(int i = 0; i < lineShapes.Length; i++)
        {
            lineShapes[i] = new LineShape();
        }
    }

    public override void Show(Graphics g)
    {
        frontRect.OutlinePen.DashStyle = OutlinePen.DashStyle;
        backRect.OutlinePen.DashStyle = OutlinePen.DashStyle;
        for(int i = 0; i < lineShapes.Length; i++)
```

```
                                {
    lineShapes[i].OutlinePen.DashStyle = OutlinePen.DashStyle;
                                }
            int left = Math.Min(StartPoint.X, EndPoint.X);
           int right = Math.Max(StartPoint.X, EndPoint.X);
            int top = Math.Min(StartPoint.Y, EndPoint.Y);
          int bottom = Math.Max(StartPoint.Y, EndPoint.Y);
                     int width = (right - left) ;
                     int height = (bottom - top);
            int depth = (Math.Min(width, height) * 1/3);

    frontRect.StartPoint = new Point(((left + (right - depth)) /
            2), (((top + depth) + bottom) / 2));
      frontRect.EndPoint = new Point((right - depth) , bottom);

    backRect.StartPoint = new Point((right + (left + depth)) / 2,
              (top + (bottom - depth)) / 2 );
        backRect.EndPoint = new Point(right, bottom - depth);


      lineShapes[0].StartPoint = new Point(left, top + depth);
       lineShapes[0].EndPoint = new Point(left + depth, top);

      lineShapes[1].StartPoint = new Point(right - depth, top +
                           depth);
          lineShapes[1].EndPoint = new Point(right, top);

        lineShapes[2].StartPoint = new Point(left, bottom);
      lineShapes[2].EndPoint = new Point(left + depth, bottom -
                           depth);

    lineShapes[3].StartPoint = new Point(right - depth, bottom);
     lineShapes[3].EndPoint = new Point(right, bottom - depth);

                        frontRect.Show(g);
                         backRect.Show(g);
              for(int i = 0; i < lineShapes.Length; i++)
                                {
                        lineShapes[i].Show(g);
                                }
                              }
                  public override void Dispose()
                                {
```

```
                frontRect.Dispose();
                backRect.Dispose();
            for(int i = 0; i < lineShapes.Length; i++)
                                {
                lineShapes[i].Dispose();
                                }
                   base.Dispose();
                                }
                               }
```

ALineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLineShape

```
                using System.Drawing.Drawing2D;

                              namespace Lab4;
                               public class
ALineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLineShape : Shape
                                {
                        private const int Radius = 16;
                 private EllipseShape firstCircle = new EllipseShape();
                 private EllipseShape secondCircle = new EllipseShape();
                        private LineShape line = new LineShape();

               private void DrawEnclosingCircle(Graphics g, Point center,
                                EllipseShape circle)
                                        {
                 circle.StartPoint = new Point(center.X - Radius, center.Y -
                                        Radius);
                  circle.EndPoint = new Point(center.X + Radius, center.Y +
                                        Radius);
                                   circle.Show(g);
                                        }
                        public override void Show(Graphics g)
                                        {
                    line.OutlinePen.DashStyle = OutlinePen.DashStyle;
                  firstCircle.OutlinePen.DashStyle = OutlinePen.DashStyle;
                 secondCircle.OutlinePen.DashStyle = OutlinePen.DashStyle;
                              line.StartPoint = StartPoint;
                                line.EndPoint = EndPoint;
                                      line.Show(g);
                    DrawEnclosingCircle(g, StartPoint, firstCircle);
                    DrawEnclosingCircle(g, EndPoint, secondCircle);
                                        }
                              public override void Dispose()
```

```
            {
        firstCircle.Dispose();
        secondCircle.Dispose();
            line.Dispose();
            base.Dispose();
                }
            }
```

Shape

```
namespace Lab4;
public abstract class Shape : IDisposable
                {
    public Point StartPoint = new Point();
     public Point EndPoint = new Point();
public Pen OutlinePen = new Pen(Color.Black);
      public bool IsPreview {get; set;}
    public abstract void Show(Graphics g);
        public virtual void Dispose()
                {
            OutlinePen.Dispose();
                }
            }
```

Descriptor

```
namespace Lab4;
public class Descriptor
                {
    public string Name {get;}
    public Bitmap Icon {get;}
public Func<Shape> Create {get;}
public Descriptor(string name, Bitmap icon, Func<Shape> create)
                {
            Name = name;
            Icon = icon;
        Create = create;
                }
            }
```

Editor

```
namespace Lab4;
 using System.Drawing.Drawing2D;
public class Editor : IDisposable
                {
```

```
public enum ShapeType
{
Point,
Line,
Rectangle,
Ellipse
}
public enum OnMouseUpResult
{
ShapeAdded,
ArrayIsFull,
NothingHappend
}
private Descriptor currentShapeType =
ShapesDescriptor.Descriptors[0];
private const int MaxShapes = 103;
private Shape[] shapes;
public Editor()
{
shapes = new Shape[MaxShapes];
}
private int shapeCount = 0;
private bool isDrawing = false;
private Point currentStart;
private Point currentEnd;
public void OnMouseDown(Point point)
{
isDrawing = true;
currentStart = point;
currentEnd = point;
}
public bool OnMouseMove(Point point)
{
if (isDrawing)
{
currentEnd = point;
}
return isDrawing;
}
public OnMouseUpResult OnMouseUp()
{
if (!isDrawing)
{
```

```
                return OnMouseUpResult.NothingHappend;
                                }

                        isDrawing = false;

                 if (shapeCount >= shapes.Length)
                                {
                  return OnMouseUpResult.ArrayIsFull;
                                }
                   Shape newShape = CreateShape();
                 newShape.StartPoint = currentStart;
                   newShape.EndPoint = currentEnd;
                    shapes[shapeCount] = newShape;
                             shapeCount++;
                 return OnMouseUpResult.ShapeAdded;
                               }
                   private Shape CreateShape()
                               {
                  return currentShapeType.Create();
                               }
               public void DrawShapes(Graphics g)
                               {
                for (int i = 0; i < shapeCount; i++)
                                 {
                            shapes[i].Show(g);
                                 }
                            if (isDrawing)
                                 {
                               DrawTemp(g);
                                 }
                               }
              private void DrawTemp(Graphics g)
                               {
          using (Shape previewShape = CreateShape())
                               {
                     previewShape.IsPreview = true;
                 previewShape.StartPoint = currentStart;
                   previewShape.EndPoint = currentEnd;
        previewShape.OutlinePen.DashStyle = DashStyle.Dash;
                        previewShape.Show(g);
                                 }
                               }
      public  void SelectShapeType(Descriptor type)
```

```
                {
            currentShapeType = type;
                }
        public void Dispose()
                {
    for(int i = shapeCount - 1; i >= 0; i--)
                    {
                shapes[i].Dispose();
                    }
                }
            }
```

Form1

```
            namespace Lab4;

    public partial class Form1 : Form
                {
            private Editor editor;
        private Descriptor currentShapeType =
        ShapesDescriptor.Descriptors[0];
    private void SelectShapeType(Descriptor type)
                {
            Text = "Lab4 - " + type.Name;
            editor.SelectShapeType(type);
                }
            private void CreateMenu()
                {
        MenuStrip menuStrip = new MenuStrip();
  ToolStripMenuItem fileMenu = new ToolStripMenuItem("Файл");
        ToolStripMenuItem objectsMenu = new
      ToolStripMenuItem("Об'єкти");
ToolStripMenuItem helpMenu = new ToolStripMenuItem("Довідка");

            Controls.Add(menuStrip);
          MainMenuStrip = menuStrip;
        menuStrip.Items.Add(fileMenu);
       menuStrip.Items.Add(objectsMenu);
        menuStrip.Items.Add(helpMenu);

       fileMenu.Click += FileMenu_Click;
       helpMenu.Click += HelpMenu_Click;
```

```
        foreach (Descriptor descriptor in ShapesDescriptor.Descriptors)
                                        {
                            ToolStripMenuItem item = new
                    ToolStripMenuItem(descriptor.Name);
                                item.Tag = descriptor;
                            item.Click += ShapeSelector_Click;
                           objectsMenu.DropDownItems.Add(item);
                                        }
                                    }
                        private void CreateShapesBar()
                                    {
                       ToolStrip shapesBar = new ToolStrip();
                    shapesBar.ImageScalingSize = new Size(24, 24);

                                Controls.Add(shapesBar);

        foreach (Descriptor descriptor in ShapesDescriptor.Descriptors)
                                        {
                             ToolStripButton item = new
                       ToolStripButton(descriptor.Name);
                  item.DisplayStyle = ToolStripItemDisplayStyle.Image;
                           item.ToolTipText = descriptor.Name;
                                item.Tag = descriptor;
                             item.Image = descriptor.Icon;
                            item.Click += ShapeSelector_Click;
                                shapesBar.Items.Add(item);
                                        }
                                    }
                                public Form1()
                                    {
                                InitializeComponent();
                        ClientSize = new Size(1080, 720);
                           this.DoubleBuffered = true;
                              editor = new Editor();
                                CreateShapesBar();
                                   CreateMenu();
                        SelectShapeType(currentShapeType);

                        this.MouseDown += Form_MouseDown;
                        this.MouseMove += Form_MouseMove;
                          this.MouseUp += Form_MouseUp;
                             this.Paint += Form_Paint;
                         this.FormClosed += Form_Closed;
```

```
                }
        private void Form_MouseDown(object? sender, MouseEventArgs e)
                {
                    editor.OnMouseDown(e.Location);
                }
        private void Form_MouseMove(object? sender, MouseEventArgs e)
                {
                  if (editor.OnMouseMove(e.Location))
                    {
                      Invalidate();
                    }
                }
         private void Form_MouseUp(object? sender, MouseEventArgs e)
                {
                    switch (editor.OnMouseUp())
                    {
                  case Editor.OnMouseUpResult.ShapeAdded:
                      Invalidate();
                      break;
                 case Editor.OnMouseUpResult.ArrayIsFull:
                    MessageBox.Show("Shapes Array is full");
                      Invalidate();
                      break;
                    }
                }
          private void Form_Paint(object? sender, PaintEventArgs e)
                {
                    Graphics g = e.Graphics;
                    editor.DrawShapes(g);
                }
        private void ShapeSelector_Click(object? sender, EventArgs e)
                {
            ToolStripItem item = (ToolStripItem)sender!;
              Descriptor type = (Descriptor)item.Tag!;
                    SelectShapeType(type);
                }
           private void FileMenu_Click(object? sender, EventArgs e)
                {
               MessageBox.Show("there is something, but not here");
                }
           private void HelpMenu_Click(object? sender, EventArgs e)
                {
          MessageBox.Show("Choose Objects option to draw some shapes");
```

```
        }
    private void Form_Closed(object? sender, FormClosedEventArgs e)
        {
            editor.Dispose();
        }
    }
```

IconShapes

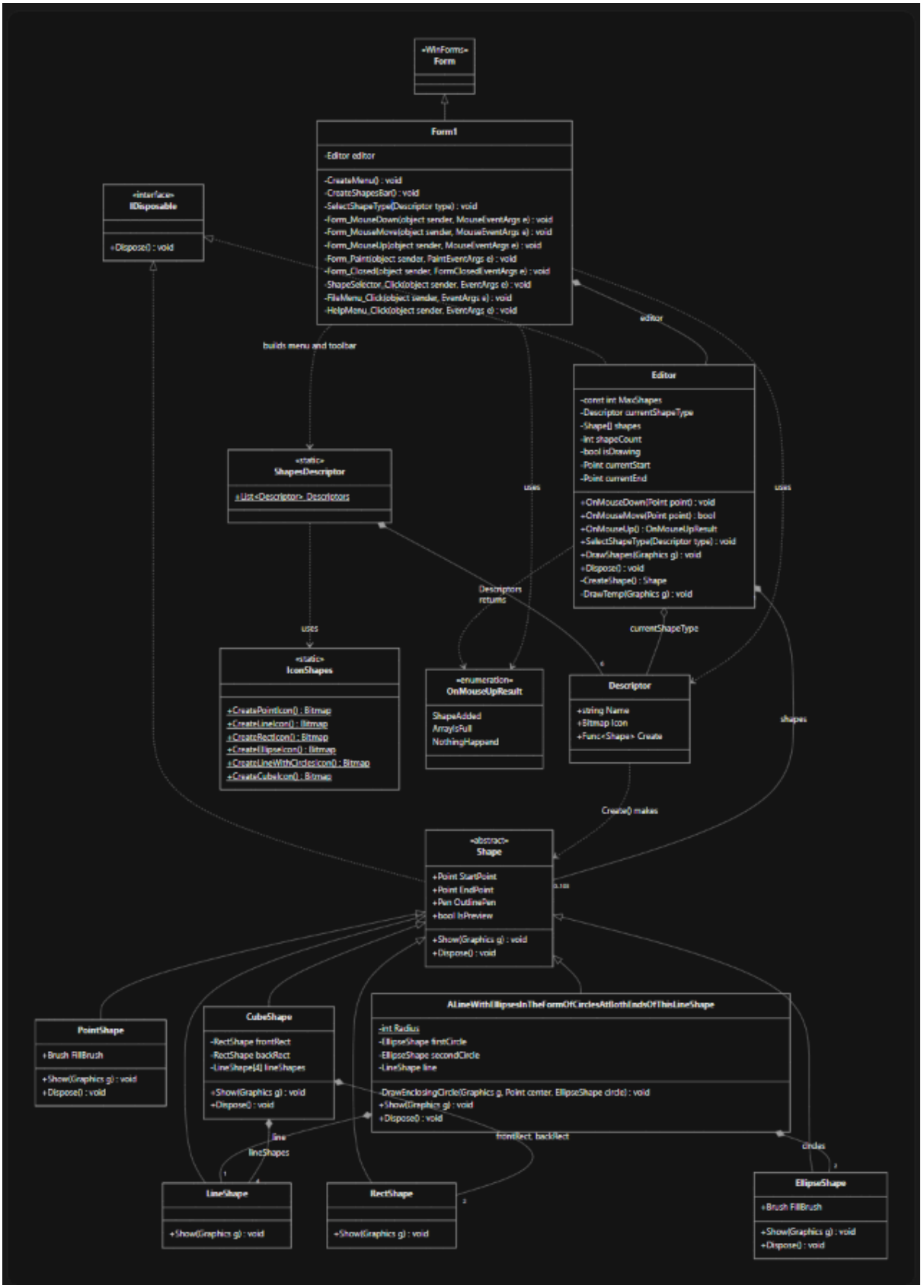
```
namespace Lab4;
static class IconShapes
{
    public static Bitmap CreatePointIcon()
    {
        Bitmap pointBmp = new Bitmap(24,24);
        using (Graphics g = Graphics.FromImage(pointBmp))
        using (Brush brush = new SolidBrush(Color.Black))
        {
            g.FillEllipse(brush, 8, 8, 8, 8);
        }
        return pointBmp;
    }
    public static Bitmap CreateRectIcon()
    {
        Bitmap rectBmp = new Bitmap(24,24);
        using (Graphics g = Graphics.FromImage(rectBmp))
        using (Pen pen = new Pen(Color.Black))
        {
            g.DrawRectangle(pen, 4, 6, 16, 12);
        }
        return rectBmp;
    }
    public static Bitmap CreateEllipseIcon()
    {
        Bitmap ellipseBmp = new Bitmap(24,24);
        using (Graphics g = Graphics.FromImage(ellipseBmp))
        using (Pen pen = new Pen(Color.Black))
        {
            g.DrawEllipse(pen, 2, 5, 20, 14);
        }
        return ellipseBmp;
    }
    public static Bitmap CreateLineIcon()
    {
```

```
                    Bitmap lineBmp = new Bitmap(24,24);
                using (Graphics g = Graphics.FromImage(lineBmp))
                     using (Pen pen = new Pen(Color.Black))
                                    {
                           g.DrawLine(pen, 3, 3, 20, 20);
                                    }
                                return lineBmp;
                                    }
                             public static Bitmap
  CreateALineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLineIcon()
                                    {
                                   Bitmap
    aLineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLineBmp = new
                                Bitmap(24,24);
                                 using (Graphics g =
Graphics.FromImage(aLineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisL
                                ineBmp))
                     using (Pen pen = new Pen(Color.Black))
                                    {
                           g.DrawLine(pen, 3, 3, 20, 20);
                          g.DrawEllipse(pen, 0, 0, 6, 6);
                         g.DrawEllipse(pen, 17, 17, 6, 6);
                                    }
                                   return
      aLineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLineBmp;
                                    }
                    public static Bitmap CreateCubeIcon()
                                    {
                      Bitmap CubeBmp = new Bitmap(24,24);
                 using(Graphics g = Graphics.FromImage(CubeBmp))
                      using(Pen pen = new Pen(Color.Black))
                                    {
                        g.DrawRectangle(pen, 0, 6, 12, 12);
                        g.DrawRectangle(pen, 6, 0, 12, 12);
                           g.DrawLine(pen, 0, 6, 6, 0);
                          g.DrawLine(pen, 12, 6, 18, 0);
                          g.DrawLine(pen, 0, 18, 6, 12);
                         g.DrawLine(pen, 12, 18, 18, 12);
                                    }
                                return CubeBmp;
                                    }
                                  }
```

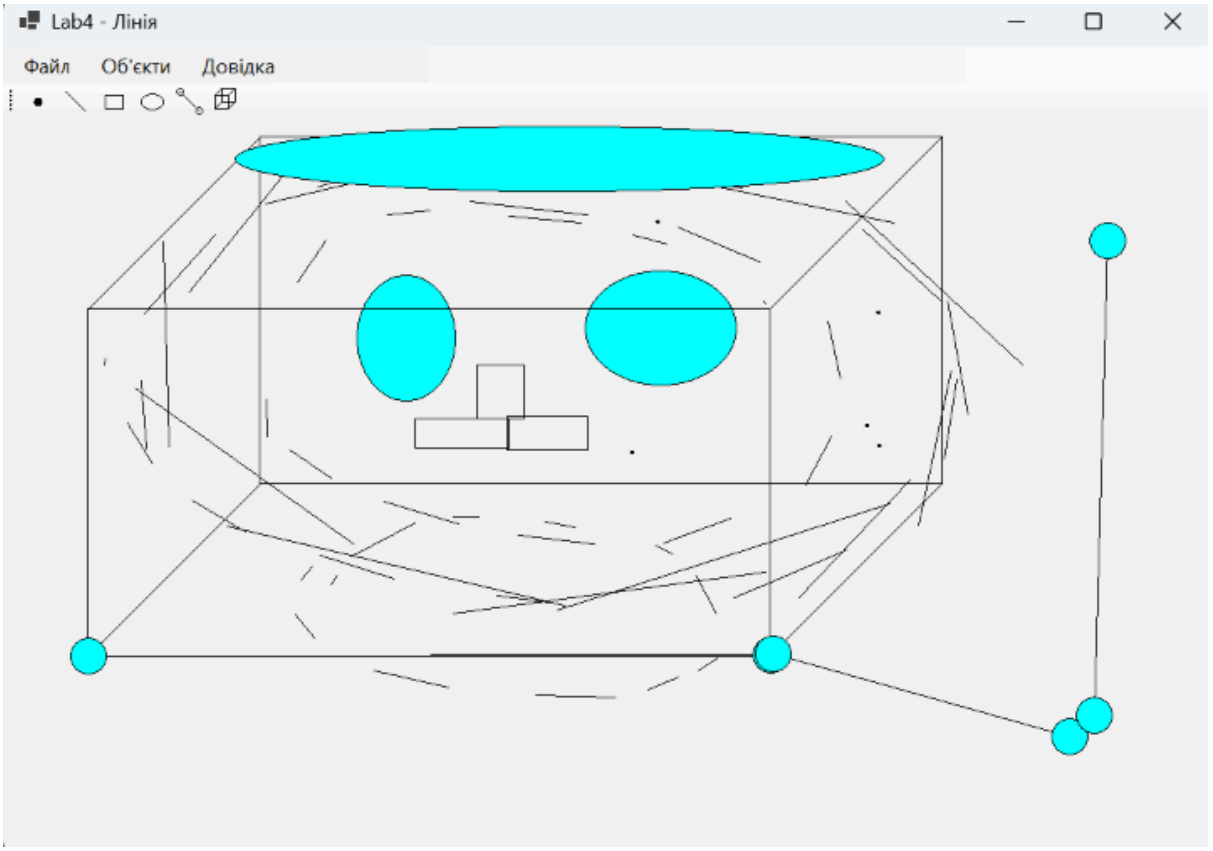
ShapeDescriptor

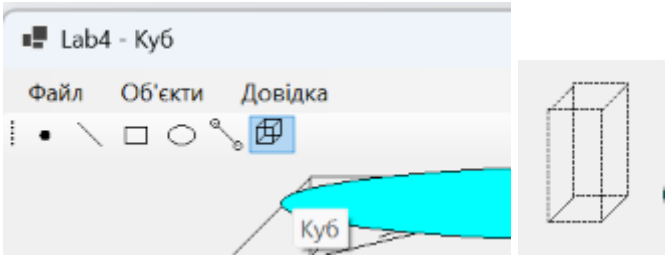
```
namespace Lab4;
public static class ShapesDescriptor
{
    public static readonly List<Descriptor> Descriptors = new
    List<Descriptor>
    {
        new Descriptor("Крапка" ,IconShapes.CreatePointIcon(), () =>
        new PointShape()),
        new Descriptor("Лінія" ,IconShapes.CreateLineIcon(), () => new
        LineShape()),
        new Descriptor("Прямокутник",IconShapes.CreateRectIcon(), () =>
        new RectShape()),
        new Descriptor("Еліпс", IconShapes.CreateEllipseIcon(), () =>
        new EllipseShape()),
        new Descriptor("Лінія з еліпсами у формі кола на обох кінцях
        цієї лінії",
IconShapes.CreateALineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLin
eIcon(), () => new
   ALineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLineShape()),
        new Descriptor("Куб", IconShapes.CreateCubeIcon(), () => new
        CubeShape())
    };
}
```

Діаграма класів


«WinForms»
Form
Form1
-Editor editor
-CreateMenu() : void
-CreateShapesBar() : void
-SelectShapeType(Descriptor type) : void
-Form_MouseDown(object sender, MouseEventArgs e) : void
-Form_MouseMove(object sender, MouseEventArgs e) : void
-Form_MouseUp(object sender, MouseEventArgs e) : void
-Form_Paint(object sender, PaintEventArgs e) : void
-Form_Closed(object sender, FormClosedEventArgs e) : void
-ShapeSelector_Click(object sender, EventArgs e) : void
-FileMenu_Click(object sender, EventArgs e) : void
-HelpMenu_Click(object sender, EventArgs e) : void
«interface»
IDisposable
+Dispose() : void
builds menu and toolbar
editor
Editor
-const int MaxShapes
-Descriptor currentShapeType
-Shape[] shapes
-int shapeCount
-bool isDrawing
-Point currentStart
-Point currentEnd
+OnMouseDown(Point point) : void
+OnMouseMove(Point point) : bool
+OnMouseUp() : OnMouseUpResult
+SelectShapeType(Descriptor type) : void
+DrawShapes(Graphics g) : void
+Dispose() : void
-CreateShape() : Shape
-DrawTemp(Graphics g) : void
«static»
ShapesDescriptor
+List<Descriptor> Descriptors
uses
uses
Descriptors
returns
currentShapeType
uses
«static»
IconShapes
+CreatePointIcon() : Bitmap
+CreateLineIcon() : Bitmap
+CreateRectIcon() : Bitmap
+CreateEllipseIcon() : Bitmap
+CreateLineWithCirclesIcon() : Bitmap
+CreateCubeIcon() : Bitmap
«enumeration»
OnMouseUpResult
ShapeAdded
ArrayIsFull
NothingHappend
Descriptor
+string Name
+Bitmap Icon
+Func<Shape> Create
shapes
Create() makes
«abstract»
Shape
+Point StartPoint
+Point EndPoint
+Pen OutlinePen
+bool IsPreview
+Show(Graphics g) : void
+Dispose() : void
PointShape
+Brush FillBrush
+Show(Graphics g) : void
+Dispose() : void
CubeShape
-RectShape frontRect
-RectShape backRect
-LineShape[4] lineShapes
+Show(Graphics g) : void
+Dispose() : void
ALineWithEllipsesInTheFormOfCirclesAtBothEndsOfThisLineShape
-int Radius
-EllipseShape firstCircle
-EllipseShape secondCircle
-LineShape line
-DrawEnclosingCircle(Graphics g, Point center, EllipseShape circle) : void
+Show(Graphics g) : void
+Dispose() : void
line
lineShapes
frontRect, backRect
circles
LineShape
+Show(Graphics g) : void
RectShape
+Show(Graphics g) : void
EllipseShape
+Brush FillBrush
+Show(Graphics g) : void
+Dispose() : void

Скріншоти





Висновок

Під час виконання лабораторної роботи я перетворив редактор з Лаби 3 на розширювану структуру: описи типів фігур зібрано в єдиному каталозі, з якого циклом будуються меню та панель кнопок, тож додавання нової фігури не потребує змін в Editor & Form1. Множинне успадкування, якого немає в C#, замінено композицією (LineShape, EllipseShape та RectShape всередині нових фігур), а коректне знищення динамічного Editor забезпечено через IDisposable по всьому ланцюжку від форми до окремих фігур.